

Informatika 3

Prednáška 1: Úvod do predmetu a jazyka C++

Michal Mrena



ŽILinskÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

6BI0013 – Informatika 3

Predstavenie predmetu



6BI0013 – Informatika 3

- Práca s pamäťou a životným cyklom objektov
- Technická realizácia známych konceptov OOP
 - Dedičnosť, virtuálne funkcie, pretypovanie
- Generické programovanie
- Kódovanie typov a reťazcov

Vyučujúci

- **Prednášajúci:**

- Ing. Michal Mrena, PhD.
- Ing. Patrik Rusnák, PhD.
- Ing. Michal Varga, PhD.

- **Cvičiaci:**

- Ing. Miroslav Gábor, PhD. (Miroslav.Gabor@fri.uniza.sk, RA117)
- Ing. Michal Mrena, PhD. (Michal.Mrena@fri.uniza.sk, RA302)
- Ing. Patrik Rusnák, PhD. (Patrik.Rusnak@fri.uniza.sk, RA324)
- Ing. Peter Sedláček, PhD. (Peter.Sedlacek@fri.uniza.sk, RA210)
- Ing. Viliam Tavač, PhD. (viliam.tavac@uniza.sk, RA123)
- Ing. Michal Varga, PhD. (Michal.Varga@fri.uniza.sk, RA324)

Náplň prednášok

1. Úvod do predmetu a jazyka C++
2. Typy a ich reprezentácia
3. Pamäťový priestor
4. OOP v jazyku C++
5. Polymorfizmus
6. Hodnotová sémantika

Náplň prednášok

- 7. Šablóny tried a funkcií
- 8. Štandardná knižnica jazyka C++
- 9. Funkcie vyššieho rádu
- 10. Kódovanie reťazcov
- 11. Jazyk C
- 12. Jazyk C

Hodnotenie predmetu

- Hodnotenie je zamerané na preverenie praktických zručností.
- Počas semestra (55b):
 - pozostáva z vypracovania troch zadaní (45b)
 - a z priebežnej práce počas semestra (10b).
 - **Pre prihlásenie sa na skúšku je potrebné získať aspoň 30 bodov.**
- Skúška (45b) pozostáva
 - z praktického riešenia problému (programovania) (40b)
 - a je doplnená otázkami k riešenému problému (5b).

Cvičenia

- Budeme používať nástroj **git**
- Prečítať si dokumenty:
 - Pravidlá predmetu.pdf
 - Vývojové prostredie.pdf
 - Git.pdf

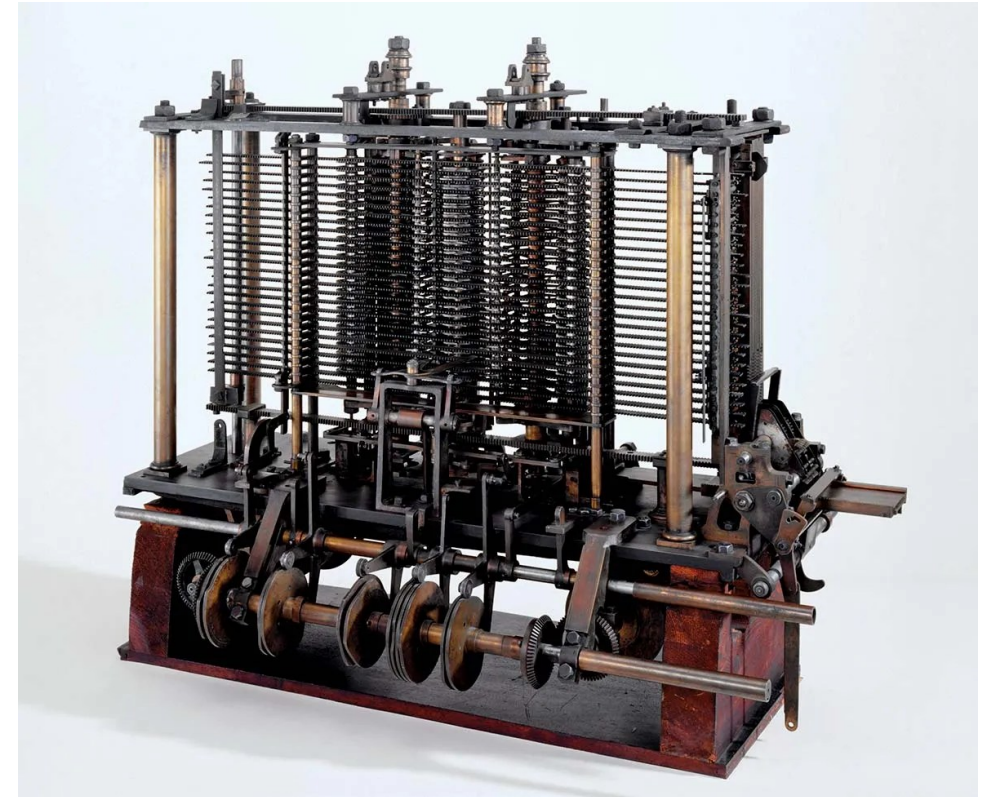
História

Ako sme sa dostali k C++



Vývoj počítačov – mechanické počítače

- **Motivácia** – automatizácia matematických výpočtov
- Vybrané osobnosti (18. – 19. stor.):
 - **Charles Babbage**
 - [Ada Lovelace](#)
 - **Joseph Jacquard**
- Ich rozhodnutia a vynálezy ovplyvnili to, ako sa dnes/donedávna programovalo



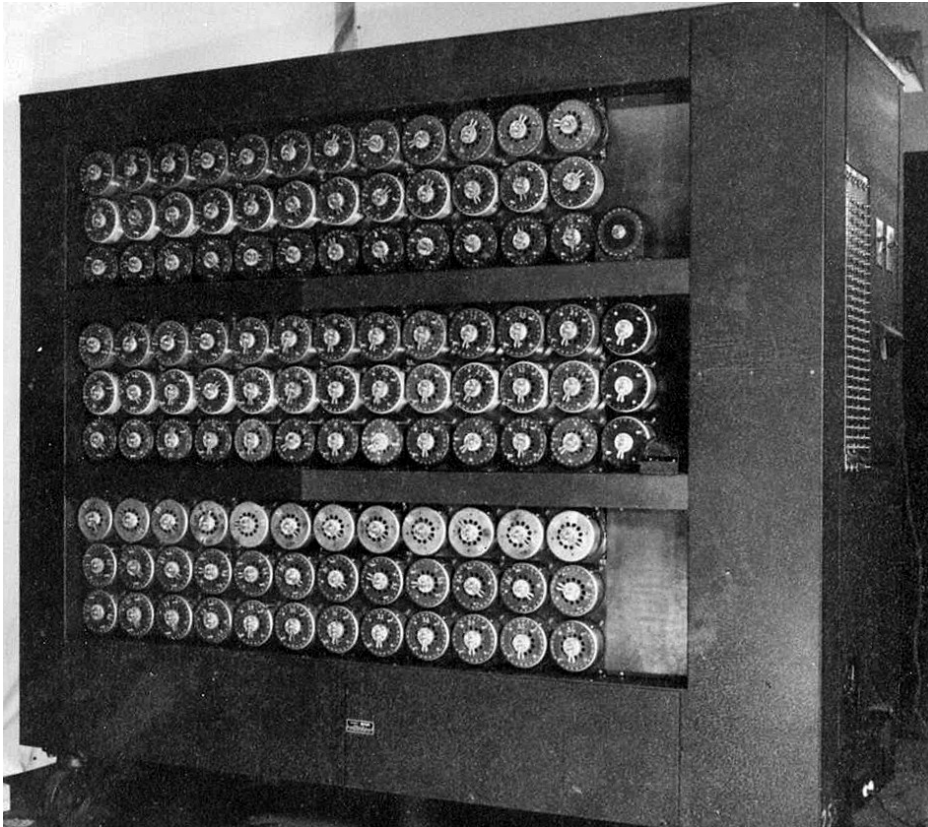
Analytical engine

<https://www.britannica.com/technology/Analytical-Engine>

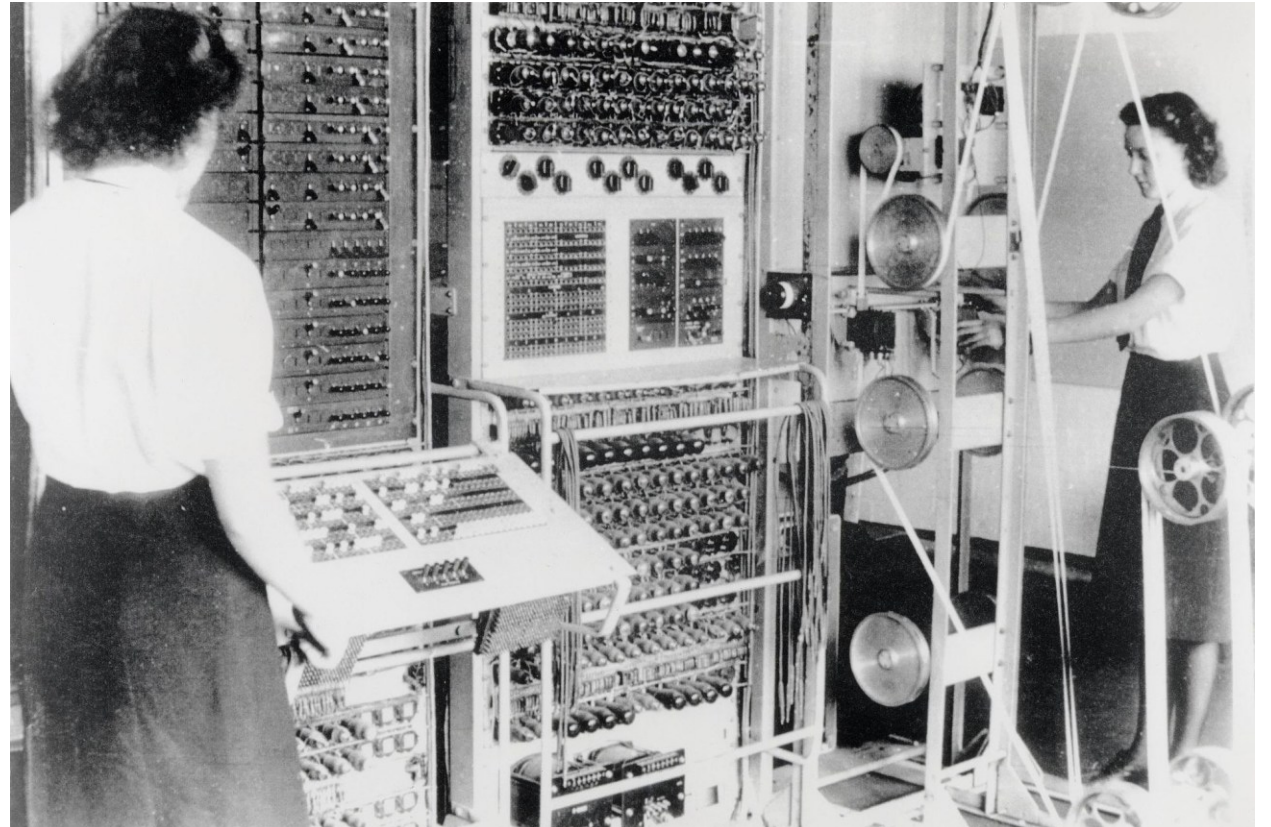
Vývoj počítačov – prechod k elektronike

- 20. storočie – poznačené vojenskými konfliktami
- **Motivácia** – výpočty balistických trajektórií, lámanie šifier, ...
- Všeobecne známe projekty a miesta:
 - Projekt Manhattan
 - Bletchley Park
- História tohto obdobia je pre informatiku veľmi dôležitá

Počítače 20. storočia



Bombe



Colossus Mark 2

Osobnosti 20. storočia



Alan Turing



Alonzo Church

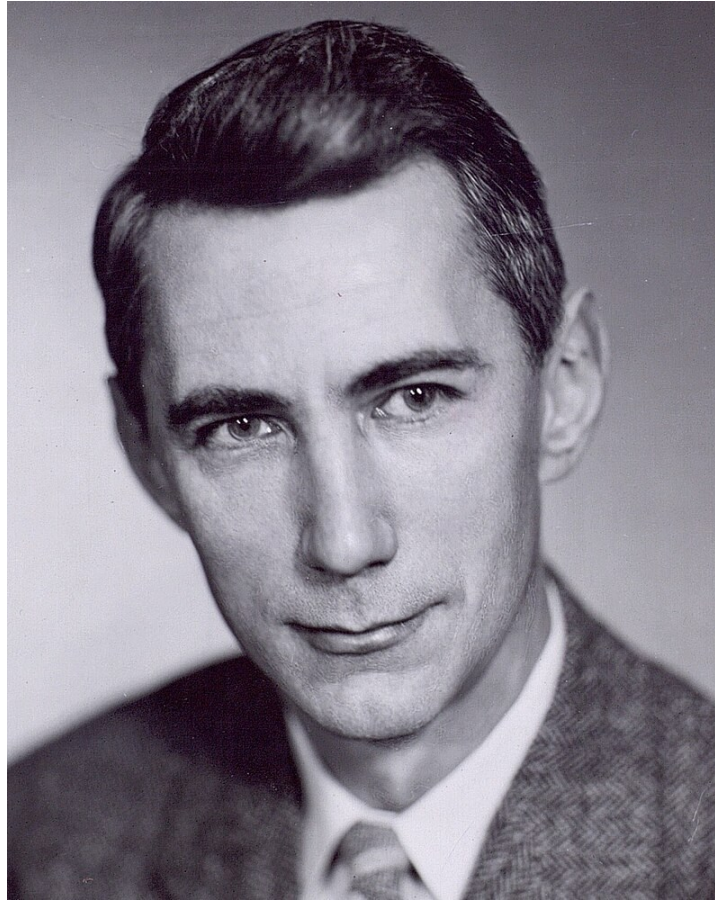


Tommy Flowers

Osobnosti 20. storočia



John von Neumann

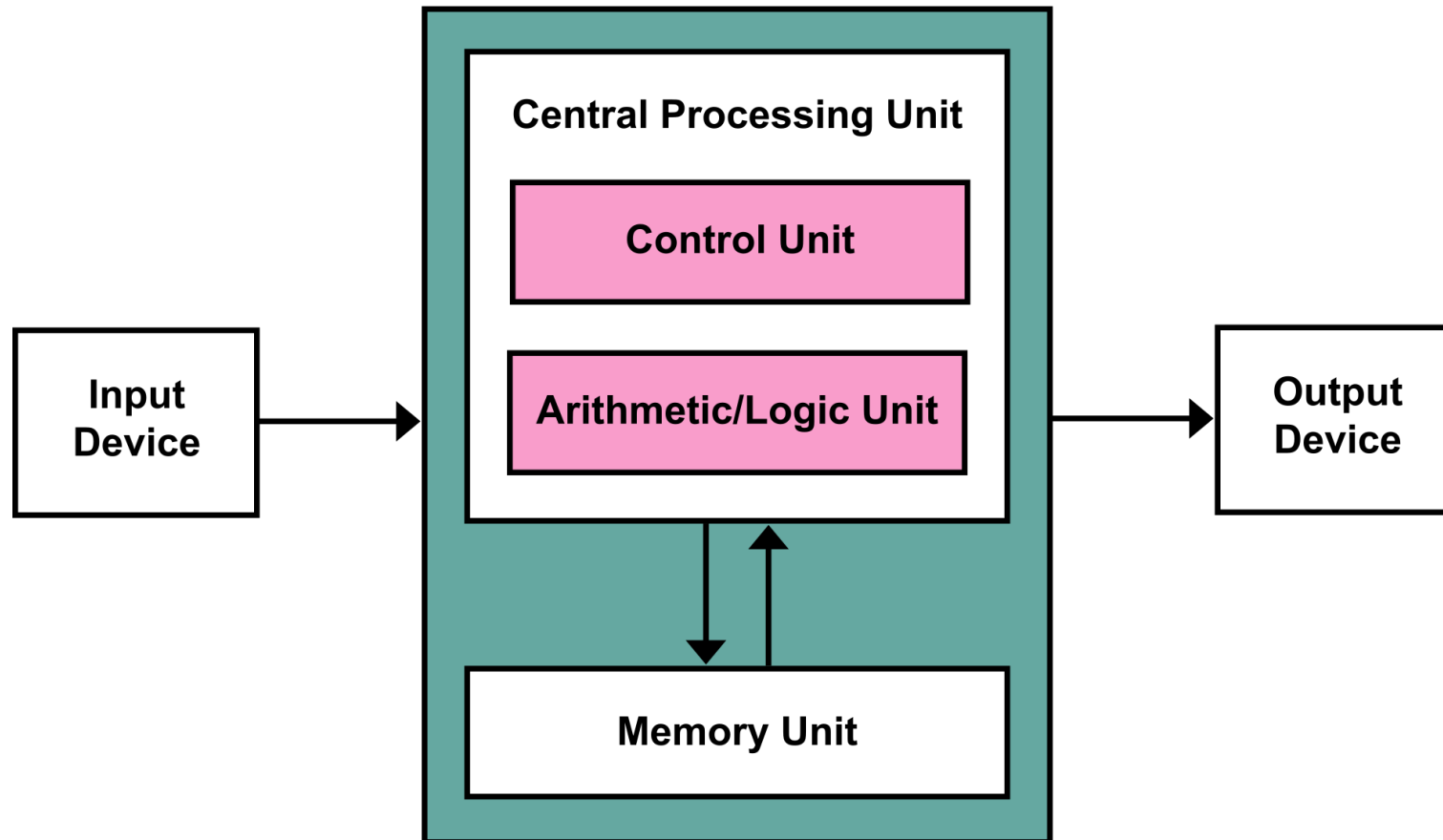


Claude Shannon



Konrad Zuse

Von Neumannova architektúra



By Kapooht - Own work, CC BY-SA 3.0,

<https://commons.wikimedia.org/w/index.php?curid=25789639>

Programovacie jazyky

Ako sme sa dostali k C++

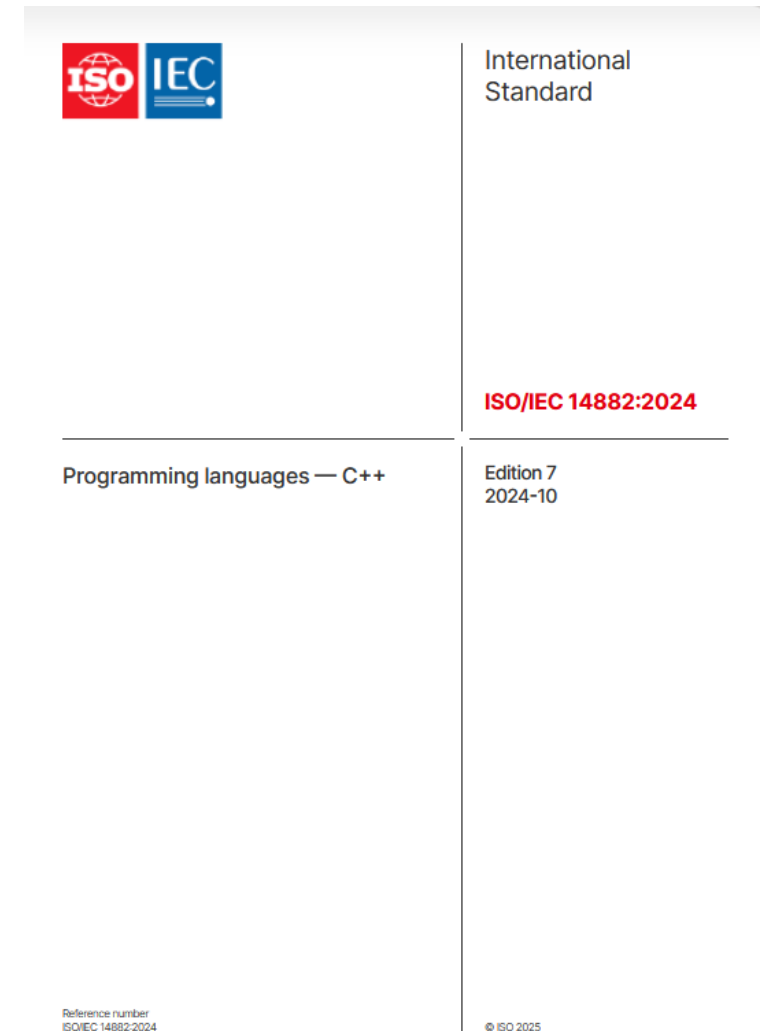


Časová os

- 1948 Plankalkül
- 1957 Fortran
- 1958 Algol
- 1960 COBOL(FLOW-MATIC 1955)
- 1960 Lisp
- 1962 Simula
- 1964 BASIC(Visual Basic 1991)
- 1970 Pascal(Delphi 1995)
- 1972 C
- 1972 Smalltalk
- 1977 Ada
- 1985 C++
- 1991 Python
- 1995 Java
- 1995 JavaScript

Programovací jazyk

- Špecifikácia
 - Dokument – štandard
 - Referenčná implementácia
- Implementácia
 - Realizácia štandardu
- Vo všeobecnosti vzťah $M:N$



Nízko-úrovňové programovacie jazyky

- Sú silne závislé na inštrukčnej sade procesora, na ktorom sa má program vykonávať
- Vysoko-úrovňové jazyky sa prekladajú do týchto jazykov

Strojový kód (dvojková sústava)

100000111100001100001010

Strojový kód (šestnástková sústava)

0x83 0xC3 0x0A

Assembler

add ebx,10

```
cmp     esi, 2147483646
mov     eax, 1
cmov     eax, esi
cmp     edi, 2147483646
cmov     eax, edi
test    esi, esi
cmov     eax, esi
test    edi, edi
cmov     eax, edi
```

<https://cpu.land/the-basics>

Vysoko-úrovňové programovacie jazyky

- Abstrahujú od špecifík počítača, na ktorom sa má program vykonávať
- Sú *štruktúrované*, avšak ľahko čitateľné pre (trénovaného) človeka

```
class Program {  
    static void Main() {  
        Console.WriteLine("Hello, world!");  
    }  
}
```

```
?- write('Hello World!'), nl.  
Hello World!  
true.
```

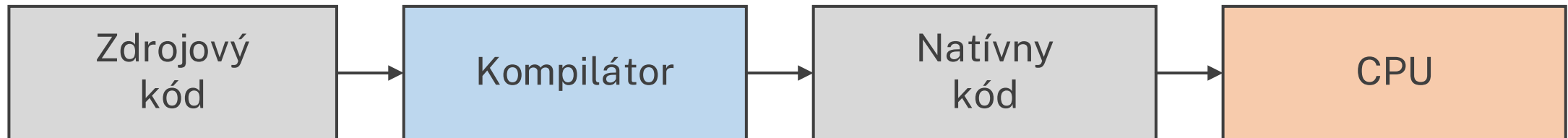
```
main :: IO ()  
main = putStrLn "Hello, World!"
```

```
program HelloWorld(output);  
begin  
    WriteLn('Hello, World!')  
end.
```

Spôsob implementácie

Kompilovaná

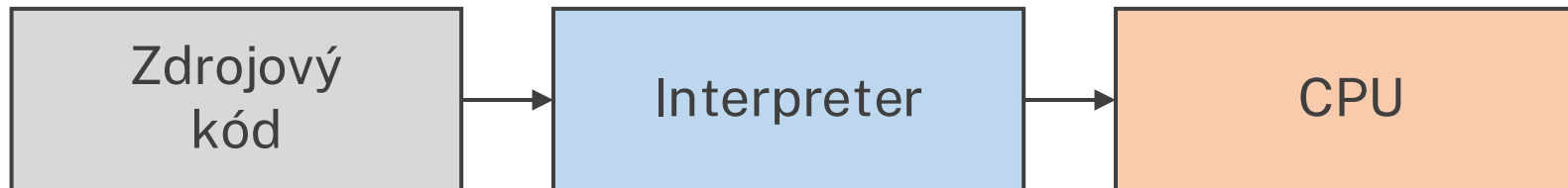
- *Zdrojový kód* – obyčajný textový **súbor**
- *Kompilátor* – **softvér** – prekladá zdrojový kód na natívny kód
- *Natívny kód* – binárny **súbor** – obsahuje inštrukcie, ktoré dokáže vykonať (interpretovať) CPU
- *CPU* – **hardvér** – vykonáva inštrukcie



Spôsob implementácie

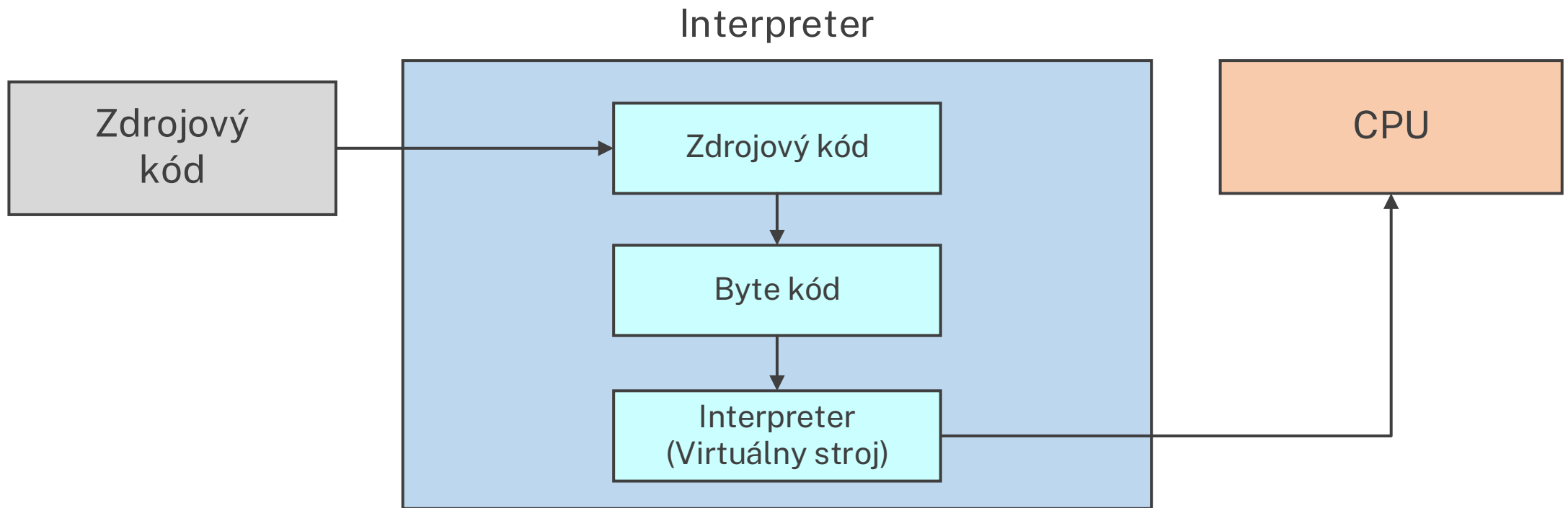
Interpretovaná

- *Interpreter* – softvér – prečíta príkaz zo zdrojového kódu a vykoná požadovanú akciu
- CPU tak môžeme považovať za hardvérový interpreter natívneho kódu
- *Implementácia jednoduchého interpretera je veľmi dobrá programátorská rozcvička*



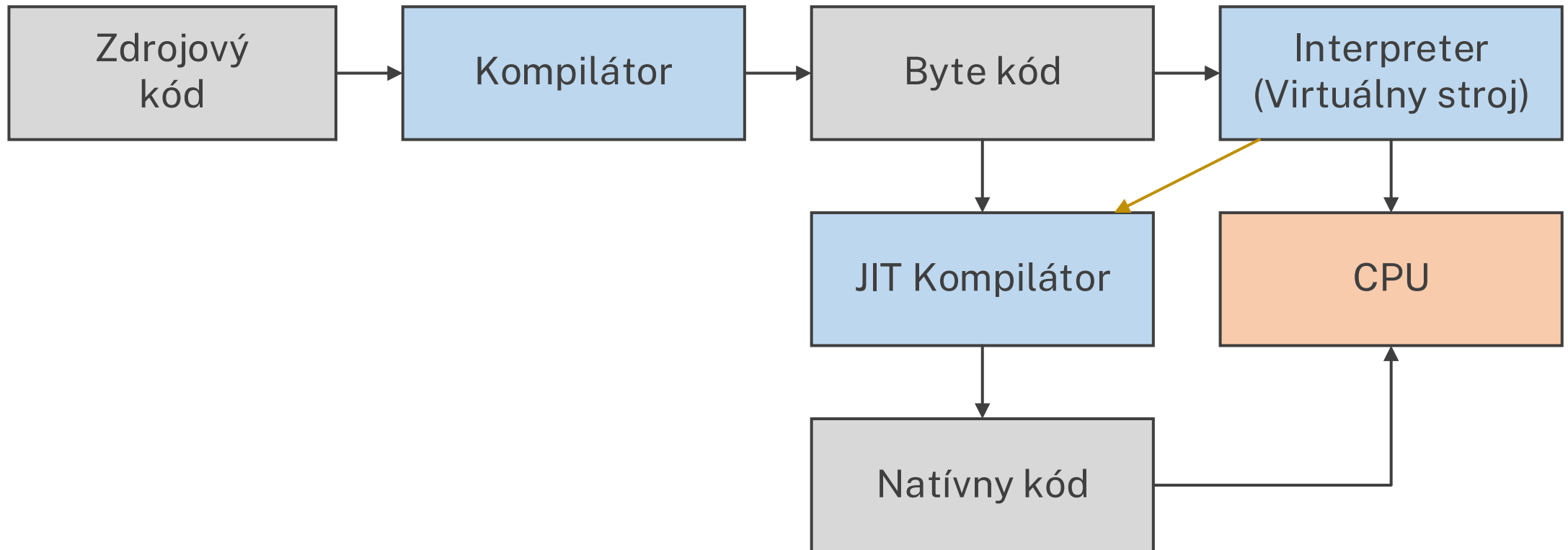
Spôsob implementácie

Interpretovaná



Spôsob implementácie

Kombinovaná



Spôsob implementácie

Kombinovaná

- *Kompilátor* – prekladá zdrojový kód na byte kód
- *Byte kód* – nízkoúrovňové inštrukcie nezávislé na CPU, ale špecifické pre virtuálny stroj
- *Virtuálny stroj* – **softvér** – vykonáva (interpretuje) byte kód, virtuálny procesor/počítač
- *JIT kompilátor* – Just In Time kompilátor – na požiadanie prekladá byte kód na natívny kód (napr. preloží často používanú funkciu)

Spustenie programov

- **Natívne kompilovaný prístup** – distribuujú sa binárne súbory (EXE, ELF), spustiteľný iba na počítači s CPU s architektúrou, pre ktorú bol kód skompilovaný (typicky C, C++, Rust, Haskell)
- **Interpretovaný prístup** – distribuujú sa zdrojové kódy (textové súbory), na počítači musí byť nainštalovaný interpreter (typicky Python, JavaScript), využíva **JIT kompilátor**
- **Kombinovaný prístup** – distribuujú sa súbory obsahujúce byte kód, na počítači musí byť nainštalovaný virtuálny stroj, na ktorom bude byte kód bežať (typicky Java, C#) , využíva **JIT kompilátor**

Jazyky

- Bez ohľadu na jazyk programujeme pre *klasické počítače* postavené na **Von Neumannovej architektúre**
- Pri sofistikovanejších optimalizáciách zohľadňujeme architektúru cieľového CPU – x86, x64, aarch64, powerpc, sparc
- Ak chceme programovaniu porozumieť do hĺbky je potrebné vidieť cez **abstrakciu**, ktorú nám ponúkajú vysokoúrovňové programovacie jazyky – chceme mať predstavu o tom, ako funguje procesor (6BI0019, 6BI0039, 6BI0003)

Jazyk C++

Jazyk C++

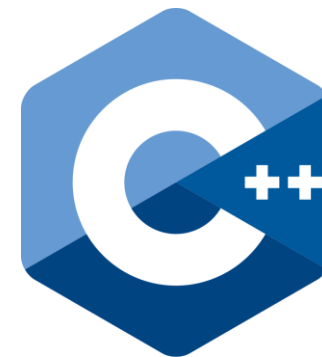


Jazyk C++

- 1985 Bjarne Stroustrup
 - Bell Labs – B, C, C++, Unix, ...
- Z počiatku iba nadstavba jazyka C
- Pretrvávajúce silné previazanie z jazykom C
 - **Napriek tomu, jazyk C/C++ neexistuje!**
- Ovplyvnený jazykom Simula
- Prvý štandard až v roku 1998 – C++98
- Patrí do „rodiny C jazykov“ – syntax podobná jazykom C, Java, C#, ...



By ICPCNews - DSC_5591,
CC BY 2.0



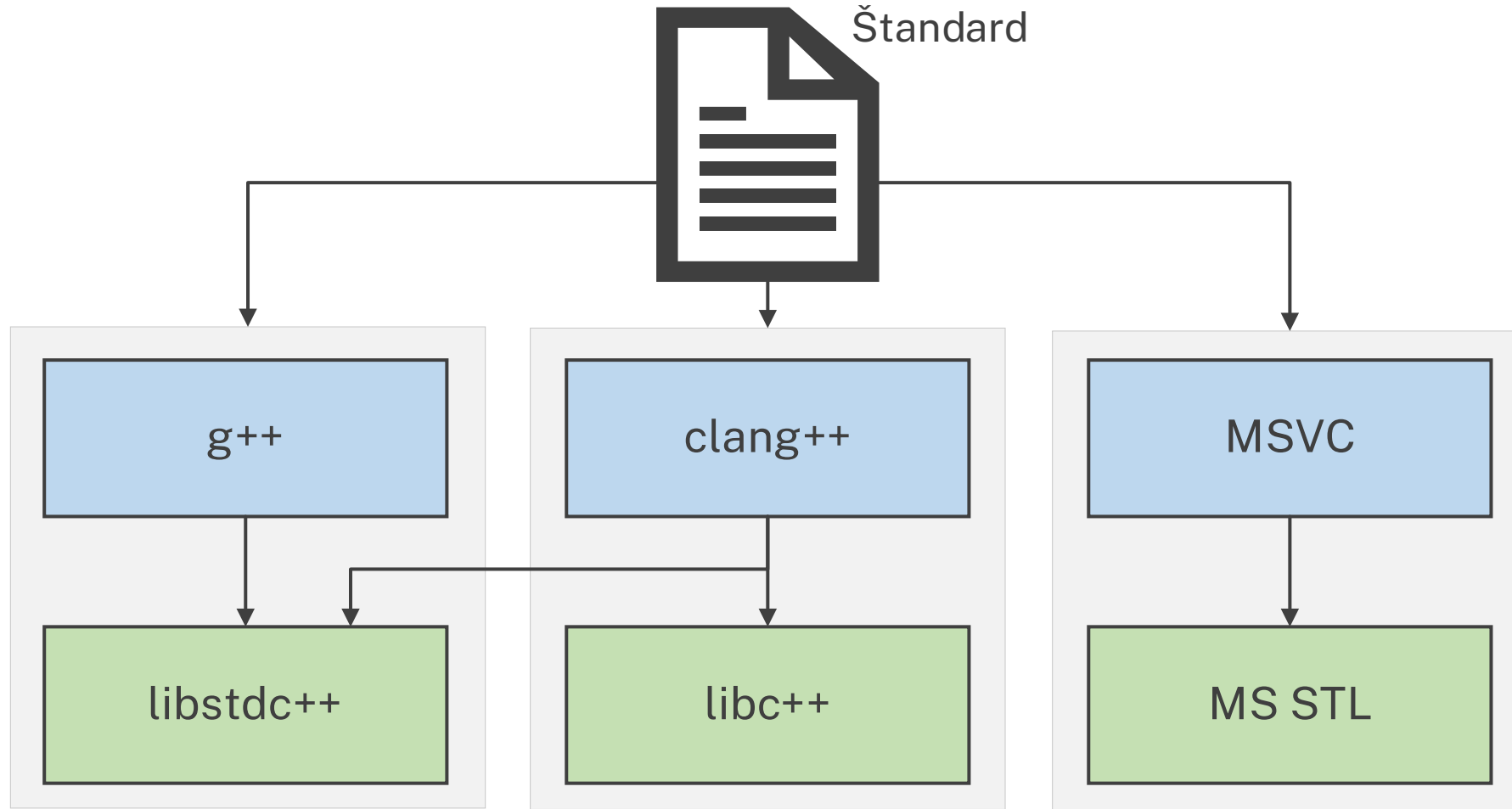
Jazyk C++ – štandardizácia

- C++93, C++03, C++11, C++14, C++17, C++20, C++23, C++26
- Názvoslovie zahŕňa rok schválenia finálnej podoby štandardu
- V súčasnosti nový štandard každé 3 roky
 - Štandard vydáva ISO
 - Za jeho prípravu je zodpovedná komisia – jej členovia pôsobia vo firmách a na univerzitách
 - Neoficiálna verzia štandardu je dostupná zdarma
- Štandard je technický dokument, neslúži ako dokumentácia
 - V komunite zaužívaná (ne)oficiálna dokumentácia:
 - <https://en.cppreference.com/w/>

Jazyk C++ – implementácia

- Implementácia = **kompilátor** + **štandardná knižnica**
- Vybrané implementácie:
 - g++ – súčasť projektu GNU
 - clang++ – súčasť projektu LLVM
 - **MSVC** – Microsoft Visual C++ – implementácia od Microsoftu, úzko prepojená s integrovaným vývojovým prostredím Visual Studio
- Mnoho ďalších implementácií – Embarcadero C++Builder, Intel C++ Compiler, Nvidia HPC C++ Compiler

Jazyk C++ – implementácia



Prvky jazyka

- Jazyk C++ je rozsiahly a komplexný
- Pri všetkom, čo si postupne ukážeme je isté **ale** – výnimka, kedy dané tvrdenie neplatí alebo okolnosti, kedy nie je pravdivé
- **Pochopenie preberaných konceptov otvorí dvere k pochopeniu mnohých iných programovacích jazykov**



Prvky jazyka

Jazyk C

- Funkcie (parametre)
- (globálne, lokálne) Premenné
- Štruktúry
- Makrá

Jazyk Java

- Balíčky
- **Triedy**
 - (statické) Atribúty
 - (statické) Metódy (parametre)
 - (lokálne) Premenné
 - Konštruktory
 - Vnorené triedy

Prvky jazyka C++

- (voľné) Funkcie (parametre)
- (globálne) Premenné
- Štruktúry
- Makrá
- ~~Balíčky~~
- Triedy
 - (statické) Atribúty (členské premenné)
 - (statické) Metódy (členské funkcie)
 - (lokálne) Premenné
 - Konštruktory
 - Vnorené triedy
- Menné priestory
- Šablóny
- Lambda funkcie
- Deštruktory

Zdrojový kód – syntax

- Syntax jazyka C++ je prebratá z jazyka C
- Patrí teda do tzv. rodiny C jazykov:
 - C, Java, C#, JavaScript, Go, ...
 - Príkazy oddelené bodkočiarkou
 - Bloky kódu organizované { }
 - Kľúčové slová: `for`, `while`, `if`, `class`, ...
 - Údajové typy: `int`, `long`, `bool`, `char`, ...

```
int int_pow(int a,int n) {  
    if (n == 0) {  
        return 1;  
    } else {  
        int h = n / 2;  
        int ph = int_pow(a,h);  
        int r = ph * ph;  
        return n&1 ? a*r : r;  
    }  
}
```

Voľná funkcia v jazyku C++

Zdrojový kód – organizácia

- Zdrojový kód je organizovaný v dvoch typoch súborov:
 - **Hlavičkové súbory**
 - Primárne obsahujú tzv. **deklarácie**
 - Textový súbor s príponou **.hpp** (.h, .hxx, ...)
 - **Zdrojové súbory**
 - Primárne obsahujú tzv. **definície**
 - Textový súbor s príponou **.cpp** (.cc, .cxx, ...)
- Početnosť vzťahu medzi **hlavičkovými** a **zdrojovými** súbormi je zvyčajne 1:1
- Prípona súboru je len nápoveda (hint) pre softvér, nemá vplyv na ich obsah
- V jazyku Java obsahoval jeden súbor **.java** definíciu jednej triedy, jeden pár **.hpp** a **.cpp** súborov môže (a zvykne) definovať viacero tried

Zdrojový kód – organizácia

- Organizácia projektu môže byť komplikovaná a rozličná naprieč projektami
- Podrobnejšie sa jej budeme venovať na cvičeniach
- Na obrázku môžeme vidieť časť projektu – hlavičkové a zdrojové súbory

```
▼ impl
  C++ ASTBuilder.cpp
  C++ ASTBuilder.hpp
  M CMakeLists.txt
  C++ ExpressionTransformer.cpp
  C++ ExpressionTransformer.hpp
  C++ NodeMapper.cpp
  C++ NodeMapper.hpp
  C++ StatementTransformer.cpp
  C++ StatementTransformer.hpp
```

Deklarácia

- Deklarácia špecifikuje, že daná entita (napr. trieda, funkcia) existuje, ale poskytuje len minimálnu nutnú informáciu o tom, ako vyzerá
- Jedna entita môže byť deklarovaná ľubovoľný počet krát

```
int int_pow(int a,int n);
```

Deklarácia funkcie `int_pow`, ktorá preberá dva parametre typu `int` a vracia hodnotu typu `int`

```
class Person;
```

(Pred-)Deklarácia triedy `Person`*

```
extern double radius;
```

Deklarácia globálnej premennej `radius` typu `double`

* V prípade tried je rozdiel medzi deklaráciou a definíciou menej intuitívny

Definícia

- Definícia je deklarácia, ktorá plne špecifikuje deklarovanú entitu
- Jedna entita môže byť definovaná (najviac) jedenkrát
 - **One Definition Rule (ODR)**

```
int int_pow(int a,int n) {  
    if (n == 0) {  
        return 1;  
    } else {  
        int h = n / 2;  
        int ph = int_pow(a,h);  
        int r = ph * ph;  
        return n&1 ? a*r : r;  
    }  
}
```

```
class Person {  
public:  
    int get_birthyear();  
  
private:  
    int m_birthyear;  
};
```

```
double radius = 2.71;
```

Deklarácia a definícia

- Deklarácií jednej entity (triedy, funkcie, premennej, ...) môže byť ľubovoľný počet
- Jedna entita však môže byť definovaná najviac jedenkrát (zvyčajne je to práve jedenkrát)
- Najčastejšie sa s týmto konceptom stretneme pri chybách pri preklade spôsobených viacnásobnou alebo chýbajúcou definíciou

Jazyk C++

Proces prekladu



Proces prekladu

- Proces prekladu pozostáva z niekoľkých fáz, ktoré za nás *nástroje* automaticky spúšťajú
- Znalosť tohto procesu však pomáha s ladením chýb a zefektívňuje proces vývoja
- Zjednodušene si popíšeme nasledovné fázy:
 - Predspracovanie
 - Kompilácia
 - Linkovanie

Príklad

- Preklad ukážeme na jednoduchom príklade, ktorý obsahuje päť súborov:
 - Person – jednoduchá trieda
 - person.hpp
 - person.cpp
 - Utils – všeobecné pomocné funkcie
 - utils.hpp
 - utils.cpp
 - Main – vstupný bod programu
 - main.cpp

Person

```
#ifndef PERSON_HPP  
#define PERSON_HPP
```

```
class Person  
{
```

```
public:
```

```
    Person(int birthyear);
```

```
    int getBirthyear();
```

```
private:
```

```
    int m_birthyear;
```

```
};
```

```
#endif
```

person.hpp

deklarácia
konštruktora

definícia
konštruktora

deklarácia
metódy

definícia
metódy

definícia
členskej
premennej

```
#include "person.hpp"
```

```
Person::Person(int birthyear) :  
    m_birthyear(birthyear)
```

```
{  
}
```

```
int Person::getBirthyear()
```

```
{
```

```
    return m_birthyear;
```

```
}
```

person.cpp

Utils

```
#ifndef UTILS_HPP
#define UTILS_HPP

/**
 * Umocni cislo a na cislo n.
 */
int int_pow(int a, int n);

#endif
```

utils.hpp

deklarácia
voľnej
funkcie

definícia
voľnej
funkcie

```
#include "utils.hpp"

int int_pow(int a, int n)
{
    if (n == 0) {
        return 1;
    } else {
        int h = n / 2;
        int ph = int_pow(a, h);
        int r = ph * ph;
        return n & 1 ? a * r : r;
    }
}
```

utils.cpp

Main

```
#include "person.hpp"
#include "utils.hpp"

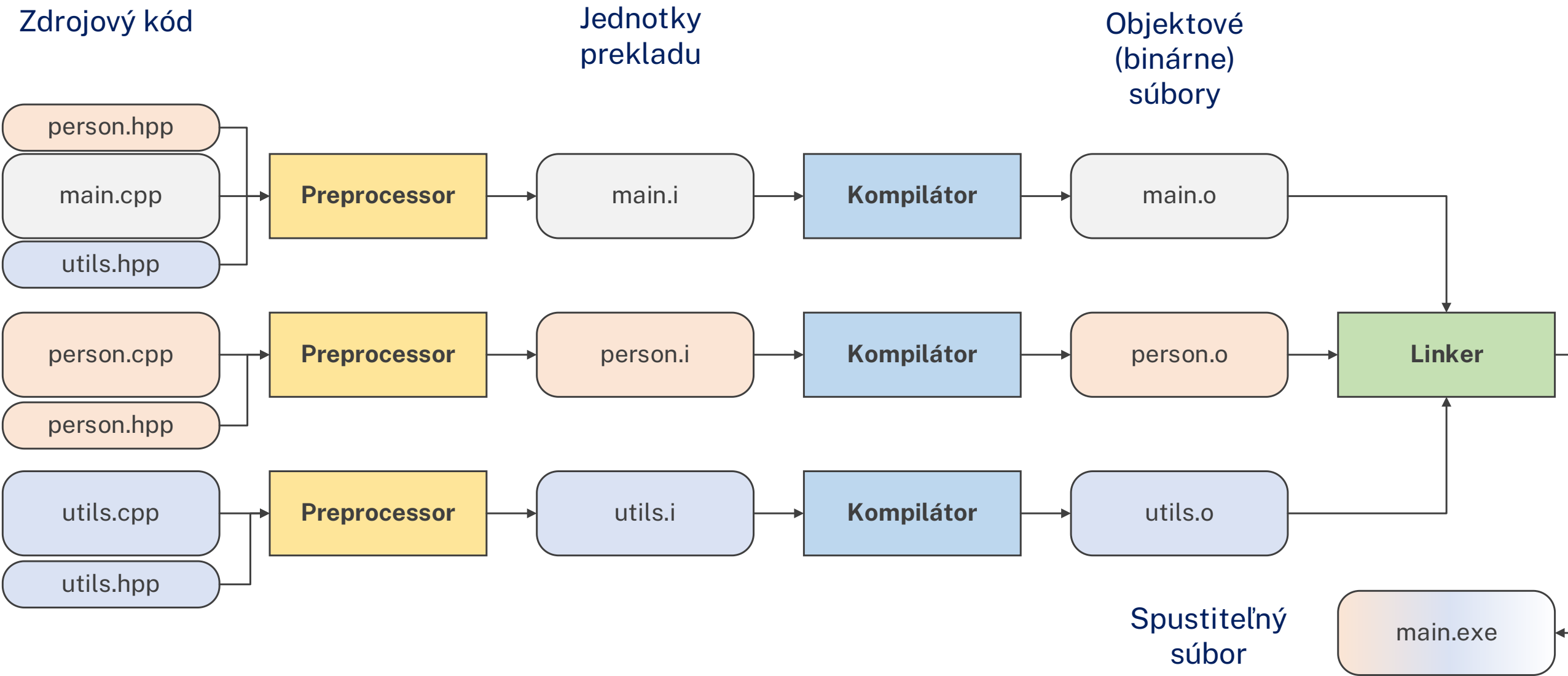
#include <iostream>

int main(int argc, char** argv)
{
    // Vytvorenie instance triedy Person.
    Person* person = new Person(1990);

    // Praca s instanciou a pomocnou funkciou.
    int by = person->getBirthyear();
    int sby = int_pow(by, 2);
    std::cout << sby << "\n";

    // Zmazanie instance.
    delete person;
}
```

main.cpp



Predspracovanie – Preprocesor

- Zabezpečuje relatívne jednoduché spracovanie textového súboru
- Riadi sa tzv. **direktívami preprocesora**
- Direktívy začínajú znakom #
- Budeme využívať nasledovné:
 - `#include`
 - `#define`
 - `#ifndef ... #endif`
- Pred spustením preprocesora sa komentáre nahradia medzerou

Preprocessor – #include

- Vloží obsah textového súboru
- Bežne sa používa na vkladanie hlavičkových súborov s definíciami
- Ide o pomerne starý a jednoduchý mechanizmus
- Novšie jazyky využívajú sofistikovanejší prístup
- Jazyk C++ má ambíciu nahradiť tento mechanizmus modulmi

Preprocessor – #include

```
#include "utils.hpp"

int int_pow(int a, int n) {
    if (n == 0) {
        return 1;
    } else {
        int h = n / 2;
        int ph = int_pow(a,h);
        int r = ph * ph;
        return n & 1 ? a * r : r;
    }
}
```

utils.cpp

```
int int_pow(int a,int n);

int int_pow(int a, int n) {
    if (n == 0) {
        return 1;
    } else {
        int h = n / 2;
        int ph = int_pow(a,h);
        int r = ph * ph;
        return n & 1 ? a * r : r;
    }
}
```

utils.i (utils.cpp po predspracovaní)

Preprocessor – #include

- Direktívna `#include` je nasledovaná **relatívnou/absolútnou** cestou k súboru, ktorý sa má vložiť, napr.
 - `#include "utils.hpp"` – v tomto prípade ide o relatívnu cestu, súbor `utils.hpp` musí byť v rovnakom priečinku ako spracovávaný súbor
 - `#include <algorithm>` – v tomto prípade ide o vloženie hlavičkového súboru štandardnej knižnice, pri použití `<>` kompilátor prehľadáva preddefinované priečinky (napr.
`C:\msys64\ucrt64\include\c++\15.2.0`)

Preprocessor – #define

- Definuje makro – identifikátor a voliteľný zoznam hodnôt, ktorý sa makro v nasledujúcom kóde nahrádza
- Často sa využíva v jazyku C, napr. na definovanie konštánt
- V jazyku C++ sa snažíme využiť bezpečnejšie alternatívy – napr. šablóny
- Využíva sa pri tzv. **podmienenom preklade** a pokročilom **generovaní zdrojového kódu** – v kombinácii s inými direktívami

Preprocessor – #define

```
#include "person.hpp"
#include "utils.hpp"

#include <iostream>

#define BEGIN {
#define END }

int main() BEGIN
    // Vytvorenie instance triedy Person
    Person* person = new Person(1990);

    int by = person->get_birthyear();
    int sby = pow(by, 2);
    std::cout << sby << "\n";

    // Zmazanie instance
    delete person;
END
```

main.cpp

<obsah vložených súborov>

```
int main() {
    Person* person = new Person(1990);

    int by = person->get_birthyear();
    int sby = pow(by, 2);
    std::cout << sby << "\n";

    delete person;
}
```

main.i (main.cpp po predspracovaní)

Preprocessor – #ifndef .. #endif

- Slúži na podmienené vynechanie/ponechanie časti súboru
- Ide o párovú direktívu – vždy musí byť ukončená direktívou #endif
- Ide o skrátenú formu všeobecnejšej direktívy #if:
 - #if !defined(SOME_MACRO) -> #ifndef SOME_MACRO
- Ak makro **nie je definované**, ponechá v súbore obsah medzi direktívami
- Hodnota macra **nie je dôležitá**, zvyčajne definujeme iba názov bez hodnoty
- Budeme ju využívať na zabránenie viacnásobného vloženia rovnakého hlavičkového súboru
- Podporuje aj úplné vetvenie – direktíva #else

Preprocessor – #ifndef .. #endif

```
void assert_equals(int a, int b) {  
#ifndef NO_DEBUG  
    if (a != b) {  
        throw std::exception(  
            "Assertion failed!");  
    }  
#endif  
}
```

debug.cpp

```
void assert_equals(int a, int b) {  
    if (a != b) {  
        throw std::exception(  
            "Assertion failed!");  
    }  
}
```

debug.i (debug.cpp po predspracovaní)

Preprocessor – #ifndef .. #endif

```
#define NO_DEBUG

void assert_equals(int a, int b) {
#ifndef NO_DEBUG
    if (a != b) {
        throw std::exception(
            "Assertion failed!");
    }
#endif
}
```

debug.cpp

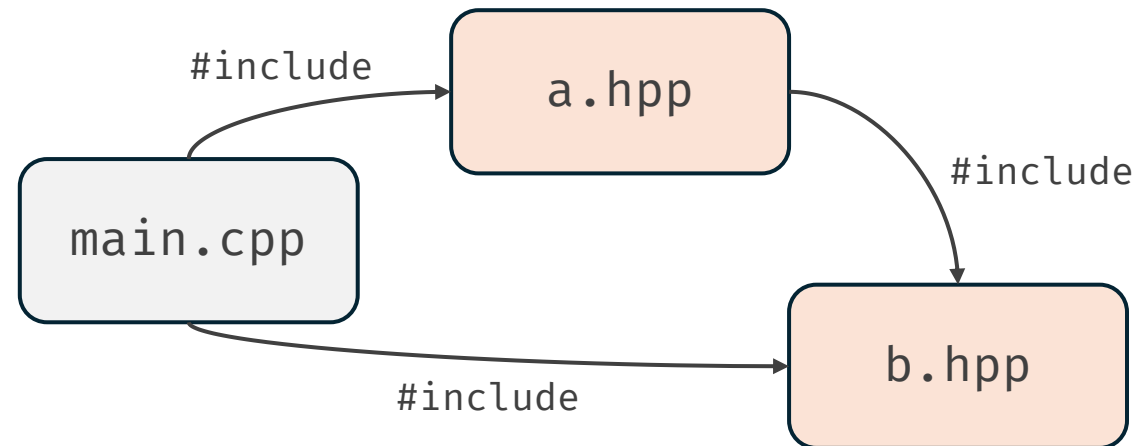
```
void assert_equals(int a, int b) {
}
```

debug.i (debug.cpp po predspracovaní)

- Testované macro môže byť (a zvyčajne aj býva) definované v inom súbore, ktorý je direktívou `#include` vložený pred vyhodnotením direktívy `#ifndef`

Ochrana vkladania hlavičkových súborov

- Hlavičkové súbory často vkladajú iné hlavičkové súbory
- Jeden hlavičkový súbor tak môže byť tranzitívne vložený viackrát
- Aj keď je viacnásobná deklarácia povolená, chceme sa duplicitu vyhnúť



Ochrana vkladania hlavičkových súborov

```
int pow(int a,int n);
```

utils.hpp

```
#include "utils.hpp"  
#include "utils.hpp"
```

```
int main() {  
    // ...  
}
```

main.cpp

```
int pow(int a,int n);  
int pow(int a,int n);
```

```
int main() {  
    // ...  
}
```

main.i (main.cpp po
predspracovaní)

Ochrana vkladania hlavičkových súborov

```
#ifndef UTILS_HPP
#define UTILS_HPP
int pow(int a,int n);
#endif
```

utils.hpp

```
#include "utils.hpp"
#include "utils.hpp"

int main() {
    // ...
}
```

main.cpp

```
#ifndef UTILS_HPP
#define UTILS_HPP
int pow(int a,int n);
#endif

#include "utils.hpp"
#define UTILS_HPP
int pow(int a,int n);
#endif
```

```
int main() {
    // ...
}
```

main.cpp po
pomyselnom
čiastočnom
predspracovaní

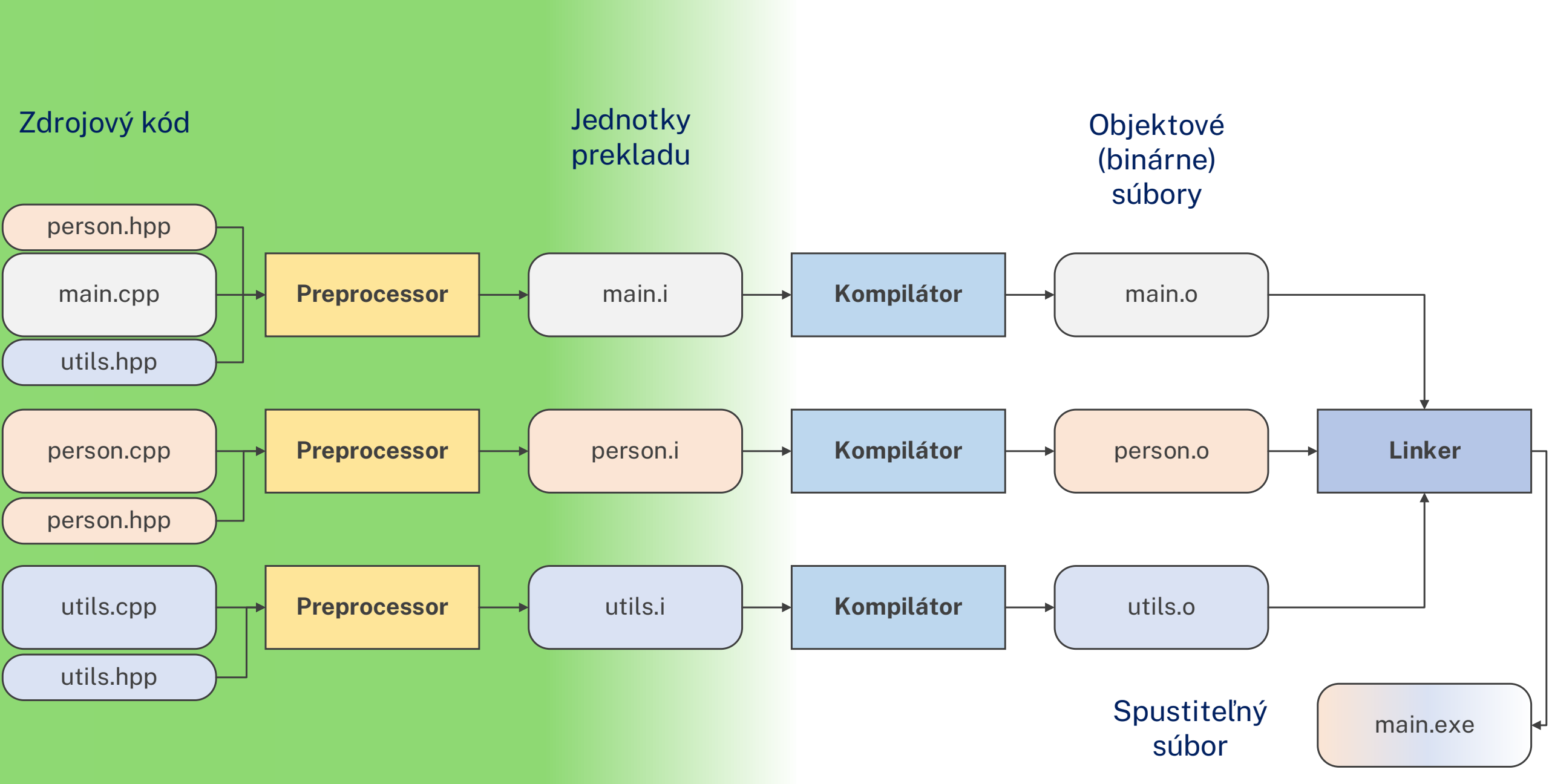
```
int pow(int a,int n);

int main() {
    // ...
}
```

main.i (main.cpp po
predspracovaní)

Ochrana vkladania hlavičkových súborov

- Názov macra je zvyčajne (kvôli prehľadnosti) totožný s názvom súboru:
 - `utils.hpp` -> `#define UTILS_HPP`
- V rozsiahlejších projektoch môže byť doplnený o cestu k súboru:
 - `libteddy/impl/node.hpp` -> `#define LIBTEDDY_IMPL_NODE_HPP`
- Ochranu pred viacnásobným vložením je možné dosiahnuť aj použitím direktívy `#pragma once` na začiatku hlavičkového súboru
- Tento spôsob však nie je štandardizovaný, hoci ho podporujú všetky väčšie kompilátory



Kompilátor

- Po fáze predspracovania vzniknú **jednotky prekladu** – pôvodné .cpp súbory spracované preprocessorom – stále sa jedná o textové súbory
- Každá jednotka prekladu sa následne kompiluje a vzniká **objektový súbor**
- Ide o **binárny súbor**, ktorý – okrem iného – obsahuje inštrukcie špecifické pre danú architektúru CPU
- Tento súbor ešte nie je spustiteľný
- Z obrázku na predchádzajúcej snímke môžeme vidieť, že na kompiláciu **nie sú potrebné definície** použitých funkcií
- Tento proces nazývame **oddelený preklad**

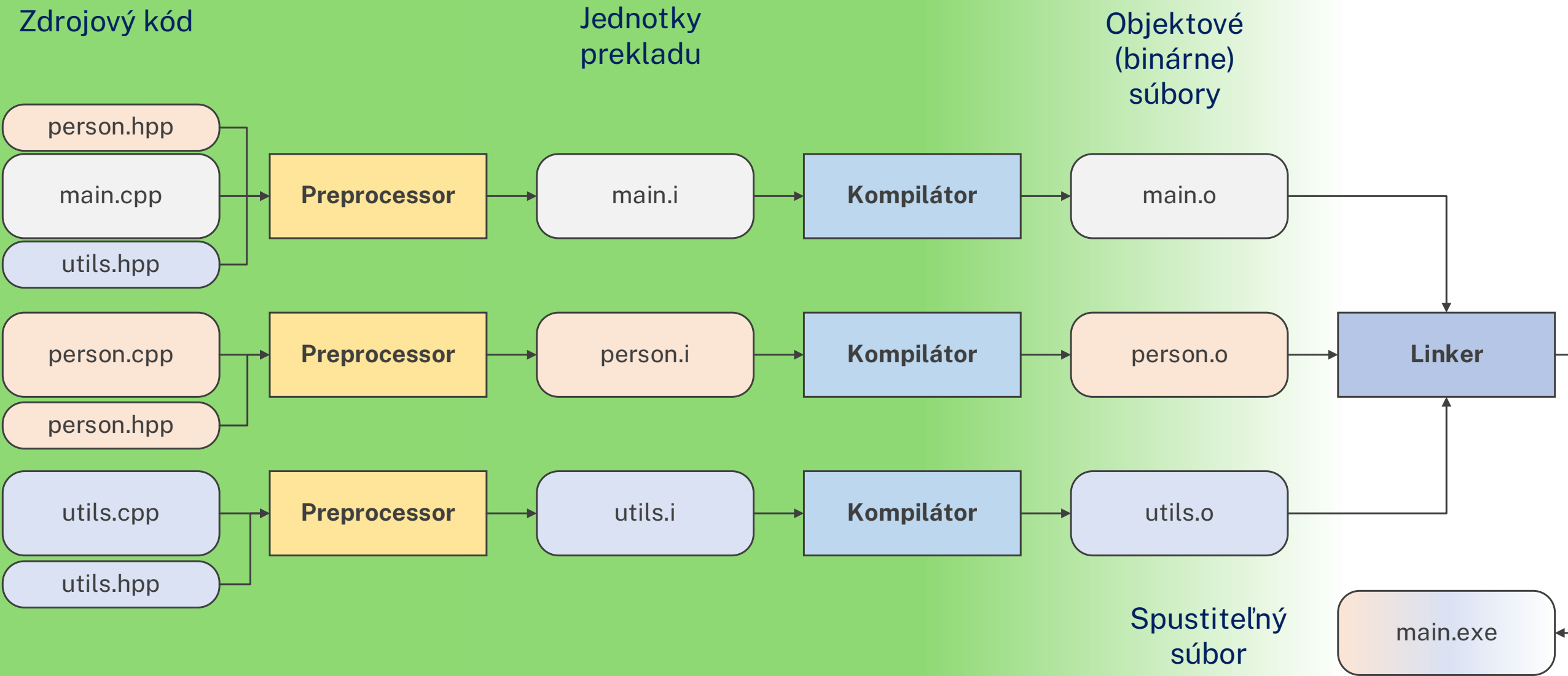
Objektový súbor

- **Názov nijako nesúvisí s OOP**
- Binárny súbor – vpravo môžeme vidieť časť jeho obsahu prepísanú do jazyka symbolických inštrukcií
- Obsahuje **volania funkcií***, ktorých inštrukcie v danom súbore **nie sú**
 - `call "pow(int, int)"`
- O následné spojenie do jedného spustiteľného súboru sa stará **linker**

```
"main":  
sub    rsp, 24  
mov    edi, 4  
mov    QWORD PTR [rsp+8], rbx  
call   "operator new(unsigned long)"  
mov    esi, 1990  
mov    rdi, rax  
mov    rbx, rax  
call   "Person::Person(int)"  
mov    rdi, rbx  
call   "Person::getBirthyear()"  
mov    esi, 2  
mov    edi, eax  
call   "pow(int, int)"  
mov    edi, OFFSET FLAT:"std::cout"  
mov    esi, eax
```

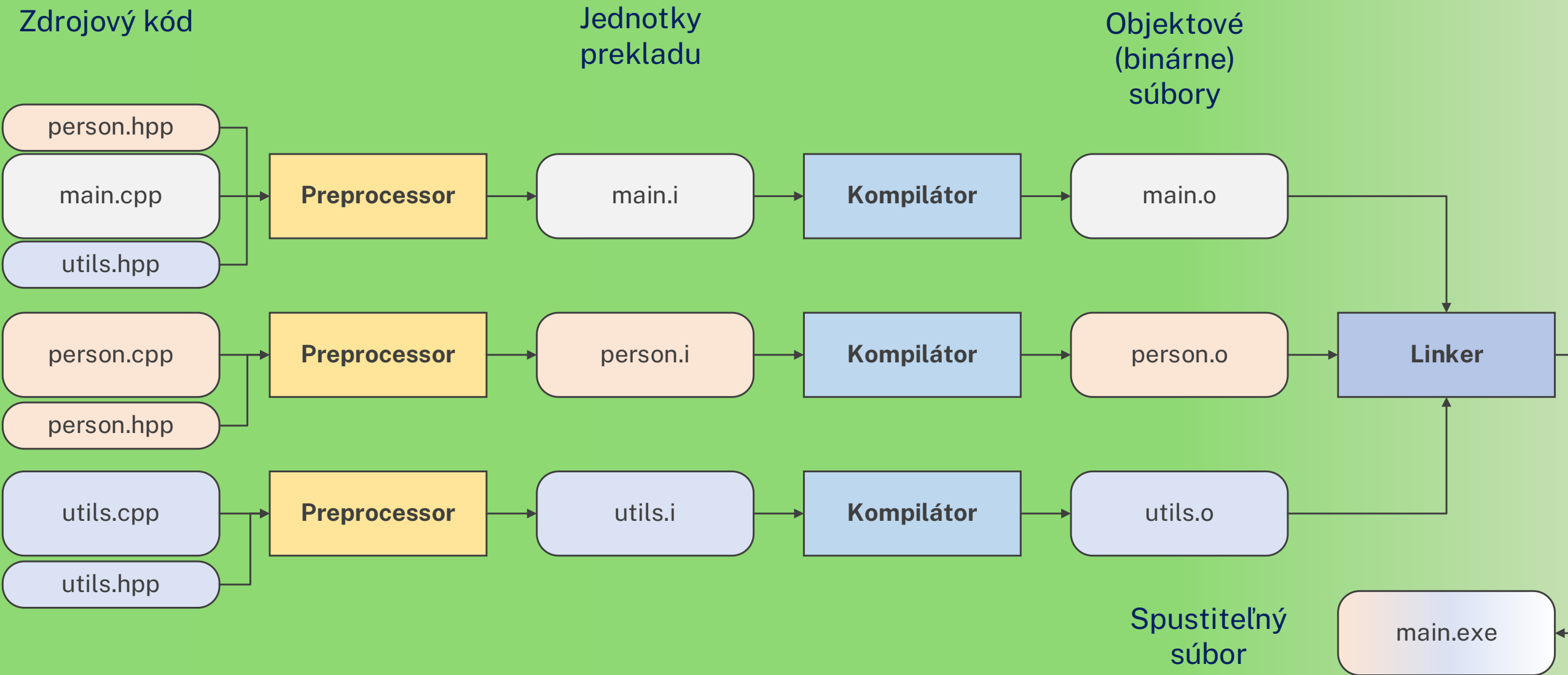
...

* Názvy funkcií nie sú v skutočnosti pekne čitateľné, ako v ukážke, ale prechádzajú transformáciou



Linker

- Jeho úlohou je spojiť niekoľko **objektových súborov** do jedného **spustiteľného súboru** (.exe) alebo knižnice (.dll, .a, .so)
 - Výsledný súbor obsahuje inštrukcie na vykonanie funkcií *
 - Jednou z úloh je priradiť volaniam funkcií miesto v súbore, na ktorom sú inštrukcie funkcie
 - V tomto bode musia existovať všetky **definície** a to práve **raz**
 - Tu často vidíme chyby s tým spojené:
 - „undefined reference to ...“
 - „multiple definition of ...“
 - Viac o linkeroch [tu](#)
- Výnimkou sú dynamicky linkované knižnice. *



Zhrnutie

- Najdôležitejšia časť prednášky je **proces prekladu**
 - Postupne sa mu budeme venovať na cvičeniach
- Prečítajte si dokument **Pravidlá predmetu.pdf**
 - V prípade nejasností sa pýtajte – nie na discorde, ale vyučujúcich

Ďakujem za pozornosť

Priestor na otázky

